

👨‍💻Clase 50 - Instalación y configuración de Kubernetes en Ubuntu Server 24.04

1. Contexto de la clase

En la **Clase 49** hemos aprendido que Kubernetes sirve para administrar aplicaciones contenedorizadas mediante recursos como **Pods**, **Deployments** y **Services**. También estudiamos el **Control Plane** y los **Worker Nodes**.

Hasta ahora, todas las prácticas se han ejecutado sobre **Ubuntu Server 24.04**, accediendo desde **VS Code mediante Remote - SSH**, y ya tenemos **Docker Engine** instalado.

En esta clase **no crearemos un clúster de alta disponibilidad con tres nodos de control y varios workers**. Primero construiremos un **clúster real de un solo nodo**, suficiente para aprender y desplegar las primeras aplicaciones.

> Un único nodo puede ejecutar los componentes del Control Plane y, al mismo tiempo, ejecutar Pods de usuario. Esto es apropiado para nuestro laboratorio, pero no ofrece tolerancia a la pérdida de ese servidor.

1.1. Qué aprenderemos

1. Elegir una distribución Kubernetes para el laboratorio.
2. Comprobar recursos y conectividad de Ubuntu.
3. Instalar y arrancar **K3s**.
4. Entender la relación entre K3s, Docker y `containerd`.
5. Comprobar el estado del clúster con `kubectl`.
6. Explorar los Pods y componentes del sistema.
7. Desplegar y exponer un primer Nginx con un manifiesto YAML.
8. Observar una recuperación automática y escalar réplicas.
9. Limpiar los recursos de la práctica sin borrar el clúster.
10. Resolver errores frecuentes.

1.2. El camino recorrido: de Docker a Kubernetes

Pregunta de entrada: si ya sabemos arrancar Nginx con `docker run` y tres servicios con `docker compose up -d`, ¿qué añade Kubernetes? Hoy comprobaremos en la práctica la **declaración del estado deseado**, la **reconciliación** y la

gestión de réplicas. En este clúster de una sola VM todavía **no** obtendremos tolerancia al fallo de la máquina.

1.3. Recuperamos la aplicación de la Clase 48

En la Clase 48 combinamos **Python**, **PostgreSQL** y tareas de **ETL** mediante Compose. Dependiendo del laboratorio, trabajamos con servicios que se descubren por nombre y con un volumen que conserva los datos de PostgreSQL cuando se eliminan y recrean contenedores.

```
CLASE 48: UNA MÁQUINA / DOCKER COMPOSE

Ubuntu Server 24.04
  └─ Docker Compose (compose.yaml)
       ├─ Python (aplicación o cliente)
       ├─ PostgreSQL (volumen de datos)
       └─ proceso ETL

CLASES 50–51+: KUBERNETES

Ubuntu Server 24.04
  └─ K3s (clúster de laboratorio)
       ├─ Deployment + Service       ← API Python
       ├─ Job / CronJob              ← ejecución ETL puntual / programada
       └─ StatefulSet + PVC          ← una posible opción para PostgreSQL
```

> **Alcance de esta clase:** el esquema Kubernetes anterior es un **mapa de aprendizaje**, no un despliegue que vayamos a ejecutar íntegramente hoy. Usaremos **Nginx** para comprobar primero que el clúster funciona; migrar Python, ETL y PostgreSQL requerirá explicar con calma imágenes, configuración, networking y persistencia. Un `StatefulSet` y un `PVC` por sí solos no garantizan alta disponibilidad ni copias de seguridad de PostgreSQL.

2. ¿Qué Kubernetes instalaremos? K3s

💡 **K3s** es una distribución ligera de Kubernetes, diseñada para facilitar su instalación y operación en entornos como laboratorios, desarrollo, edge y equipos con recursos limitados. Incluye componentes esenciales como **containerd**, **CoreDNS** y una implementación de red de Pods basada en **Flannel**. Según la configuración, también puede incluir Traefik y ServiceLB.

En esta clase elegimos K3s porque:

- Es **Kubernetes real**, no un simulador.
- Permite trabajar sobre el **Ubuntu Server que ya hemos configurado**.
- Incluye el cliente `kubectl` y configura los componentes necesarios.
- Admite ampliaciones posteriores a múltiples nodos.

Otras opciones habituales para aprender son **kind**, **minikube** o una instalación con **kubeadm**. No las mezclaremos en esta práctica: seguiremos una sola ruta para evitar confusiones.

Documentación oficial: https://docs.k3s.io/ y https://docs.k3s.io/quick-start

2.1. Arquitectura que construiremos



```
        │ │ kubelet + containerd    │ │
        │ │ Pod Nginx               │ │
        │ └─────────────────────────┘ │
        └─────────────────────────────┘
```

💡 **Diferencia respecto a la imagen conceptual de la Clase 49:** allí mostrábamos varios nodos y un control plane redundante. Aquí tanto la administración como los Pods residen en una **sola VM**. Si se apaga Ubuntu, se interrumpe todo el laboratorio.

3. Requisitos previos

3.1. Comprobar el entorno

Antes de empezar:

- Tener arrancada la máquina virtual **Ubuntu Server 24.04**.
- Poder conectarnos desde **VS Code con Remote - SSH**.
- Disponer de una conexión a Internet en Ubuntu.
- Tener un usuario con permisos para `sudo`.
- Conservar Docker Engine de las clases anteriores.

Recursos de la VM: los mínimos oficiales para un nodo K3s de tipo *server* son **2 núcleos CPU y 2 GB RAM**, sin contar cargas de trabajo. Para esta clase recomendamos **2 vCPU y 4 GB RAM o más** y suficiente almacenamiento libre (por ejemplo, **20 GB disponibles**), si el equipo anfitrión lo permite. Son recomendaciones pedagógicas, no garantías de rendimiento.

Referencia: https://docs.k3s.io/installation/requirements

3.2. Abrir una terminal remota

En VS Code, después de conectarnos a Ubuntu mediante Remote - SSH, abrir:

```
Terminal > New Terminal
```

Todos los comandos de Linux, Docker y Kubernetes siguientes se ejecutan en la terminal remota de Ubuntu, salvo que se indique expresamente Windows.

Comprobar usuario, máquina y sistema:

```
whoami
hostname
cat /etc/os-release
uname -m
```

Comprobar los recursos:

```
nproc
free -h
df -h /
```

Obtener la IP de Ubuntu:

```
hostname -I
```

La dirección depende de cada laboratorio; **no copies una IP de ejemplo como si fuera la tuya**.

3.3. Verificar Internet y preparar utilidades

```
sudo apt update
sudo apt install -y curl ca-certificates
```

Comprobar resolución DNS y acceso HTTPS:

```
getent hosts get.k3s.io
curl -I https://get.k3s.io
```

La respuesta HTTP puede variar; el objetivo es que el nombre se resuelva y la conexión se establezca.

3.4. Revisar servicios y puertos de prácticas anteriores

Si conservas Nginx u otro servicio Docker de clases previas, revísalo:

```
docker ps
sudo ss -lntup | grep -E ':(80|443|6443|30080)\b' || true
```

No detengas indiscriminadamente contenedores con datos importantes. En nuestra instalación desactivaremos los complementos **Traefik** y **ServiceLB** para simplificar el laboratorio y minimizar conflictos de puertos 80/443. El puerto **6443/tcp** del API Server debe estar disponible.

> **Seguridad:** no expongas el puerto 6443 a Internet. En nuestro laboratorio, `kubectl` se ejecutará **dentro de Ubuntu**, a través de la sesión SSH ya configurada.

4. Kubernetes no necesita ejecutar Docker Engine como runtime

En las clases 43–48 usamos Docker para gestionar contenedores:

```
Comandos Docker → Docker Engine → contenedores
```

K3s instala y utiliza por defecto **containerd**:

```
kubectl → Kubernetes API → kubelet → containerd → contenedores de los Pods
```

Por tanto:

- **No desinstalaremos Docker Engine**: seguirá disponible para prácticas Docker y Docker Compose.
- Las imágenes compatibles con OCI que construimos usando Docker pueden utilizarse en Kubernetes si el clúster puede acceder a ellas mediante un registro o una importación explícita.
- `docker ps` **no muestra necesariamente los contenedores gestionados por K3s**, porque usan un runtime y un almacenamiento de imágenes distintos.
- Para examinar recursos Kubernetes utilizaremos `kubectl`; para inspecciones avanzadas del runtime existen `k3s crictl` y `k3s ctr`, que no son necesarios en esta primera práctica.

5. Instalar K3s

5.1. Precaución antes de instalar

La instalación configura servicios y modifica el sistema operativo. En una VM de prácticas es recomendable crear una **instantánea o snapshot** si tu hipervisor lo permite.

La guía oficial utiliza un script de instalación descargado de Internet. En un entorno profesional deberíamos revisar el instalador, definir un procedimiento reproducible y fijar las versiones. **Aquí usamos el procedimiento oficial sencillo para un laboratorio controlado.**

5.2. Ejecutar la instalación

En la terminal de Ubuntu ejecuta:

```
curl -sfL https://get.k3s.io | sudo sh -s - server \
  --disable traefik \
  --disable servicelb
```

Explicación:

Parte	Función
`curl -sfL https://get.k3s.io`	Descarga el instalador oficial.
`sudo sh -s - server`	Ejecuta la instalación en modo servidor.
`--disable traefik`	No instala el controlador Ingress Traefik predeterminado.
`--disable servicelb`	No utiliza el balanceador de servicios integrado.

Los parámetros de desactivación son una **elección de laboratorio**. Un clúster de producción puede utilizar esos componentes u otras alternativas.

> **Nota:** si el comando falla, **no ejecutes repetidamente scripts a ciegas**. Lee primero la salida y consulta la sección de resolución de problemas.

5.3. ¿Qué acaba de instalarse?

De forma simplificada:

1. El servicio `k3s` se registra con `systemd`.
2. K3s arranca los componentes del servidor Kubernetes.
3. El nodo incorpora también las funciones necesarias para ejecutar Pods.
4. Se configura la red básica y el runtime de contenedores.
5. Se genera un archivo de acceso al clúster llamado **kubeconfig**.
6. Se instala el cliente de administración `kubectl` integrado con K3s.

> **Importante:** en esta instalación mononodo K3s utiliza por defecto **SQLite** como almacenamiento del estado del clúster; **no estamos montando todavía el clúster etcd replicado** que aparecía en el diagrama de alta disponibilidad de la Clase 49.

6. Comprobar el servicio Kubernetes

6.1. Estado de `k3s`

```
sudo systemctl status k3s
```

Busca un estado similar a:

```
Active: active (running)
```

Para salir del visor:

```
q
```

Si el servicio está arrancando, puede tardar un tiempo en poner todos los componentes a disposición.

6.2. Consultar los últimos registros

```
sudo journalctl -u k3s -n 50 --no-pager
```

Para seguir nuevos mensajes:

```
sudo journalctl -u k3s -f
```

Detener el seguimiento con **Ctrl + C**. Esto **no detiene** el servicio K3s.

6.3. Comprobar la versión

```
sudo k3s --version
sudo k3s kubectl version --client
```

La versión concreta puede variar según la fecha de instalación. No es necesario que coincida con una captura de pantalla de la clase.

7. Primera conexión con kubectl

7.1. ¿Qué es `kubectl`?

`kubectl` es el cliente de línea de comandos que utilizamos para comunicarnos con la API de Kubernetes.

```
Usuario
  | kubectl get nodes
  ▼
Kubernetes API Server
  |
  ▼
Información de los nodos
```

En K3s podemos emplear directamente:

```
sudo k3s kubectl get nodes
```

Una salida aproximada es:

```
NAME       STATUS   ROLES                  AGE   VERSION
dataeng    Ready    control-plane,master   ...   v1.xx.x+k3s...
```

El nombre, los roles exactos y la versión pueden diferir. **Lo importante es `Ready`.**

7.2. Configurar kubectl para nuestro usuario

El archivo kubeconfig de K3s se encuentra en:

```
/etc/rancher/k3s/k3s.yaml
```

Está protegido porque contiene **credenciales administrativas del clúster**. Para trabajar desde el usuario habitual de Ubuntu, podemos crear una copia privada:

```
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown "$(id -u):$(id -g)" ~/.kube/config
chmod 600 ~/.kube/config
```

Comprobar:

```
kubectl get nodes
```

> **Seguridad:** no publiques este archivo en GitHub, no lo incluyas en un Dockerfile y no lo copies a carpetas compartidas. **Su posesión permite administrar el clúster**. Esta copia es útil en el laboratorio; en producción debe aplicarse un sistema de identidades y permisos específico. Si cambian credenciales o certificados, puede ser necesario actualizarla.

7.3. Verificar el contexto

```
kubectl config current-context
kubectl cluster-info
```

`kubectl config current-context` muestra a qué contexto está apuntando el cliente. Antes de administrar clústeres reales, **comprueba siempre el contexto activo**.

8. Inspeccionar el primer clúster

8.1. Listar nodos

```
kubectl get nodes
kubectl get nodes -o wide
```

Algunos campos importantes:

Campo	Significado
`NAME`	Nombre del nodo.
`STATUS`	Estado del nodo (`Ready`, `NotReady`, etc.).
`ROLES`	Roles anunciados por el nodo.
`VERSION`	Versión de Kubernetes.
`INTERNAL-IP`	IP del nodo dentro de la red.

8.2. Conocer sus detalles

```
kubectl describe node "$(hostname)"
```

Si el nombre del nodo Kubernetes es diferente al hostname del sistema, obtén primero el nombre real con `kubectl get nodes` y úsalo manualmente.

En la salida veremos datos como:

- Capacidad de CPU y memoria.
- Recursos disponibles para Pods.
- Condiciones del nodo.
- Pods que están asignados al nodo.
- Eventos recientes.

8.3. Listar los Namespaces

```
kubectl get namespaces
```

Deberían aparecer namespaces del sistema como `kube-system`, además de `default` y otros creados por Kubernetes.

8.4. Ver Pods del sistema

```
kubectl get pods -n kube-system
```

Podemos observar componentes como **CoreDNS**, el proveedor local de almacenamiento y otros Pods del sistema (según versión y configuración).

```
kubectl get pods -A
```

La opción `-A` significa **todos los namespaces**.

Pregunta : ¿Por qué aparecen Pods si todavía no hemos desplegado una aplicación propia?

▼ **Respuesta:**

porque el propio clúster utiliza componentes internos que se ejecutan como Pods

9. Primeros comandos de kubectl

Comando	¿Para qué sirve?
`kubectl get nodes`	Lista los nodos del clúster.
`kubectl get pods`	Lista Pods del namespace actual.
`kubectl get pods -A`	Lista Pods de todos los namespaces.
`kubectl get namespaces`	Lista namespaces.
`kubectl get deployments`	Lista Deployments.
`kubectl get services`	Lista Services.
`kubectl describe pod NOMBRE`	Muestra detalles y eventos de un Pod.
`kubectl logs NOMBRE`	Muestra logs de un Pod.
`kubectl apply -f ARCHIVO.yaml`	Crea o actualiza los recursos descritos en un manifiesto.
`kubectl delete -f ARCHIVO.yaml`	Elimina los recursos declarados en el archivo.
`kubectl explain deployment`	Consulta la estructura y definición de ese recurso.

> **Diferencia con Docker:** `docker ps` consulta los contenedores gestionados por Docker Engine. `kubectl get pods` pregunta a Kubernetes por los **Pods** del clúster.

9.1. Puente de comandos: lo que ya sabemos hacer

Esta tabla compara **intenciones similares**, no comandos que sean intercambiables ni recursos equivalentes uno a uno:

Lo que hacíamos con Docker / Compose	Lo que comenzamos a hacer en Kubernetes	Diferencia importante
`docker ps`	`kubectl get pods -n clase50`	Kubernetes organiza contenedores en **Pods**.
`docker logs CONTENEDOR`	`kubectl logs deployment/nginx-clase50 -n clase50`	Consultamos una carga declarada en el clúster; con varias réplicas conviene seleccionar Pod y/o agregar logs.
`docker run -d --name web -p 8080:80 nginx`	`Deployment` + `Service` (`NodePort` en nuestro laboratorio)	Separamos la ejecución de la exposición en red.
`docker compose up -d`	`kubectl apply -f archivo.yaml`	Compose administra el proyecto definido; `kubectl apply` declara o actualiza recursos del clúster.
`docker compose ps`	`kubectl get deployments,pods,services -n clase50`	Vemos diferentes clases de recursos, no simplemente contenedores.
`docker compose down`	`kubectl delete -f archivo.yaml`	Elimina **solo** lo definido por los manifiestos indicados; no equivale a borrar todos los recursos de un namespace.
`docker volume ls`	`kubectl get pv,pvc -A`	PV/PVC representan almacenamiento Kubernetes, no son volúmenes Docker Engine.
`docker network ls`	`kubectl get services -A`	No son equivalentes: la conectividad entre Pods depende de la red del clúster (CNI), y los Services ofrecen descubrimiento y acceso estable.

💡 **Mini-reto:** antes de seguir, identifica en los manifiestos de las secciones 10 y 11 qué parte describe **cómo se ejecuta Nginx** y cuál describe **cómo se accede a Nginx**.

10. Primera aplicación: Nginx en Kubernetes

Objetivo: desplegar un servidor web Nginx y comprobar que Kubernetes mantiene el estado deseado.

10.1. Crear la carpeta de trabajo

```
mkdir -p ~/proyectos/kubernetes/clase50
cd ~/proyectos/kubernetes/clase50
```

Desde VS Code Remote - SSH, abre esta carpeta mediante el explorador de archivos.

10.2. Crear un namespace específico

```
kubectl create namespace clase50
```

Verificar:

```
kubectl get namespaces
```

Usaremos `-n clase50` en los siguientes comandos para no mezclar nuestra práctica con los recursos internos del clúster.

10.3. Crear `nginx-deployment.yaml`

Crear un archivo con este contenido:

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: nginx-clase50
namespace: clase50
spec:
replicas:1
selector:
matchLabels:
app: nginx-clase50
template:
metadata:
labels:
app: nginx-clase50
spec:
containers:
-name: nginx
image: nginx:1.27-alpine
ports:
-containerPort:80
resources:
requests:
cpu: 50m
memory: 32Mi
limits:
cpu: 250m
memory: 128Mi
```

Explicación de los campos:

Campo	Función
`apiVersion: apps/v1`	Versión de la API correspondiente a Deployments.
`kind: Deployment`	Tipo de recurso.
`metadata.name`	Nombre del Deployment.
`metadata.namespace`	Namespace en el que se crea.
`replicas: 1`	Número deseado de Pods.
`selector.matchLabels`	Identifica los Pods administrados por el Deployment.

Campo	Función
`template`	Plantilla utilizada para crear los Pods.
`image`	Imagen desde la que se ejecutará Nginx.
`containerPort`	Puerto declarado para el contenedor.
`resources`	Reservas y límites orientativos para el laboratorio.

Nota de versiones: `nginx:1.27-alpine` es un ejemplo de etiqueta concreta para la práctica, no una recomendación de producción actual. En proyectos reales comprobaremos mantenimiento, vulnerabilidades y procedencia de la imagen, y fijaremos una versión mantenida, idealmente por digest.

10.4. Aplicar el manifiesto

```
kubectl apply -f nginx-deployment.yaml
```

Salida aproximada:

```
deployment.apps/nginx-clase50 created
```

Comprobar:

```
kubectl get deployments -n clase50
kubectl get pods -n clase50
```

Esperamos un Pod cuyo estado termine siendo `Running` y `READY` aparezca como `1/1`.

Para esperar a que esté disponible:

```
kubectl rollout status deployment/nginx-clase50 -n clase50 --timeout=180s
```

Si no alcanza el estado esperado, consulta la sección de errores antes de avanzar.

11. Exponer Nginx con un Service

Los Pods reciben direcciones IP que pueden cambiar. Para ofrecer un acceso estable dentro de Kubernetes utilizaremos un **Service**.

11.1. Crear `nginx-service.yaml`

```yaml
apiVersion: v1
kind: Service
metadata:
name: nginx-clase50
namespace: clase50
spec:
type: NodePort
selector:
app: nginx-clase50
ports:
-name: http
port:80
targetPort:80
nodePort:30080
protocol: TCP
```

Este Service publica el puerto **30080 del nodo** y dirige el tráfico al puerto **80 del contenedor Nginx**.

> `NodePort` es apropiado para la práctica, pero no es una configuración recomendada de exposición pública de aplicaciones. Para producción estudiaremos **Ingress**, **balanceadores** y **TLS**. El puerto 30080 debe estar disponible.

11.2. Crear y comprobar el Service

```
kubectl apply -f nginx-service.yaml
kubectl get services -n clase50
```

Salida aproximada:

```
NAME            TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)
nginx-clase50   NodePort   10.43.x.x      <none>        80:30080/TCP
```

La IP `10.43.x.x` es ilustrativa. Kubernetes asignará la que corresponda.

11.3. Probar desde Ubuntu

```
curl http://127.0.0.1:30080
```

Deberíamos recibir el HTML de bienvenida de Nginx.

También puedes comprobar desde otra terminal:

```
kubectl get endpointslices -n clase50
kubectl get pods -n clase50 -o wide
```

11.4. Probar desde el navegador de Windows

En Ubuntu:

```
hostname -I
```

Si **Windows puede acceder a la IP de la máquina virtual** y el firewall permite la conexión, abre en el navegador:

```
http://IP_DE_UBUNTU:30080
```

Por ejemplo, si la IP de tu Ubuntu fuera `192.168.86.130`, la URL sería `http://192.168.86.130:30080`. **Esa IP es solo un ejemplo.**

Si la máquina virtual está en modo NAT o no admite conexiones entrantes desde Windows, usa el reenvío de puertos de VS Code Remote - SSH o el método de la siguiente sección. No desactives el firewall indiscriminadamente para que funcione la práctica.

11.5. Alternativa: `kubectl port-forward`

En la terminal remota de Ubuntu:

```
kubectl port-forward -n clase50 service/nginx-clase50 8085:80
```

En **otra terminal remota** comprueba:

```
curl http://127.0.0.1:8085
```

Para verlo en Windows, utiliza la pestaña **Ports** de VS Code Remote - SSH y reenvía el puerto remoto **8085** a tu equipo local; después abre `http://localhost:8085` en el navegador de Windows.

Importante: `kubectl port-forward` se mantiene en primer plano y se detiene con **Ctrl + C**. No confundir el `localhost` de Ubuntu con el de Windows sin establecer el túnel apropiado.

12. Experimento: Kubernetes mantiene el estado deseado

En Docker, si eliminamos un contenedor aislado manualmente, el contenedor deja de existir. En Kubernetes, un Deployment expresa cuántas réplicas queremos mantener.

12.1. Identificar el Pod

```
kubectl get pods -n clase50
```

12.2. Eliminar el Pod

Ejecutar:

```
kubectl delete pod -n clase50 -l app=nginx-clase50
```

Atención: este comando elimina los Pods que tengan esa etiqueta. En nuestra práctica hay uno.

12.3. Observar la recuperación

```
kubectl get pods -n clase50 -w
```

Observa cómo aparece otro Pod creado por el controlador del Deployment. Pulsa **Ctrl + C** para salir del seguimiento.

Pregunta: ¿Se ha recuperado el mismo Pod o Kubernetes ha creado uno nuevo?

▼ Respuesta

Respuesta: el Deployment, mediante su ReplicaSet, mantiene el número deseado creando un **Pod de reemplazo**. No es que se “resucite” el Pod eliminado.

> **Límite:** esta recuperación funciona mientras el clúster y el nodo tengan recursos suficientes. **Un clúster mononodo no puede trasladar los Pods a otro servidor si la VM falla.**

13. Experimento: aumentar el número de réplicas

Ahora vamos a pedir tres instancias de Nginx:

```
kubectl scale deployment/nginx-clase50 --replicas=3 -n clase50
```

Comprobar:

```
kubectl get deployments -n clase50
kubectl get pods -n clase50 -o wide
```

Podremos ver **tres Pods**, siempre que haya recursos suficientes.

Idea clave: aunque tengamos tres Pods, en esta instalación **los tres pueden ejecutarse en la misma VM**. Escalar réplicas **no equivale automáticamente a disponer de alta disponibilidad física**.

13.1. Volver a una réplica

```
kubectl scale deployment/nginx-clase50 --replicas=1 -n clase50
```

Observación sobre YAML y cambios declarativos

El archivo `nginx-deployment.yaml` todavía contiene `replicas: 1`. El comando de escalado cambia el estado del recurso en el clúster. En flujos declarativos, debemos mantener **sincronizados los manifiestos versionados y los cambios operativos** para evitar diferencias inesperadas al volver a aplicar una configuración.

14. Consultar logs y diagnosticar Pods

14.1. Ver el estado

```
kubectl get pods -n clase50
```

14.2. Describir un Pod sin escribir su nombre completo

```
kubectl describe pods -n clase50 -l app=nginx-clase50
```

En el resultado interesa localizar:

- La imagen utilizada.
- Los estados del contenedor.
- Las condiciones del Pod.
- Los eventos de planificación, descarga o errores.

14.3. Consultar logs del Deployment

```
kubectl logs -n clase50 deployment/nginx-clase50
```

Con seguimiento:

```
kubectl logs -n clase50 -f deployment/nginx-clase50
```

Si existen varias réplicas, el comando sobre el Deployment puede seleccionar un Pod. Para observar todos los Pods seleccionados según el comportamiento de tu versión de `kubectl`, utiliza los selectores y opciones de logs correspondientes.

14.4. Eventos del namespace

```
kubectl get events -n clase50 --sort-by=.lastTimestamp
```

Los eventos son muy útiles para diagnosticar errores de recursos, programación de Pods y descarga de imágenes.

15. Limpieza del laboratorio

15.1. Eliminar solo nuestra aplicación

```
kubectl delete -f nginx-service.yaml
kubectl delete -f nginx-deployment.yaml
```

Verificar:

```
kubectl get all -n clase50
```

15.2. Eliminar el namespace de la clase

```
kubectl delete namespace clase50
```

El clúster K3s sigue instalado y funcionando. Lo necesitaremos para la siguiente clase.

15.3. Comprobar que seguimos teniendo Kubernetes

```
kubectl get nodes
sudo systemctl is-active k3s
```

> **No ejecutes `k3s-uninstall.sh` al terminar esta clase.** Ese script **desinstala K3s** y elimina datos locales del clúster; es una operación distinta de borrar los recursos de Nginx. Además, evita comandos destructivos como `docker system prune --volumes` porque podrías perder volúmenes de prácticas anteriores.